PPMD_1.6 — Final Project Presentation

Written by Jurijus Pacalovas

Lossless Tournament Compression with Extension-as-Header Design

---

Slide 1 — Title & Problem Statement

PPMD_1.6 is a lossless compression framework built on one question: can a search over reversible byte transformations beat pure entropy coding?

Instead of applying one fixed compressor to a file, PPMD tries every one of 256 reversible transforms (or a two-step composition from 65,280 ordered pairs), compresses each candidate with a pool of modern backends, and keeps whichever combination yields the smallest round-trippable output. The winner's identity — which transform or pair was used — is not stored inside the file at all; it lives in the filename extension, so zero header bytes are added.

Core question: Can a cheap reversible pre-transform reveal redundancy that zstd-22, brotli-11, LZMA-9e, PPMd, or cmix cannot see on their own?

Slide 2 — Architectural Overview

The pipeline has four stages:

1. XML structural pre-pass (a61) — detects DOCX / ZIP / plain XML and re-encodes tags, attributes, and text into dictionary-indexed streams before any backend sees the data.

2. Single-transform sweep — applies each of a1…a256, verifies reversibility, compresses the result with the backend pool (cback), and records the size.

3. Pair tournament — composes a → b for every ordered pair, parallelised across CPU cores.

4. Forced cmix pass — a final Level-9 pass on the winner only (cmix is far too slow to run across 65,000 candidates).

The winning method is encoded in the extension:

· 0x0000..0x00FF → single transform a1..a256

· 0x0100..0xFFFF → pair index b1..b65280

The decoder reads the extension, reverse-runs the exact same transforms, and reconstructs the original bytes — with no header bytes to parse.

Slide 3 — The 256 Transformations

The transforms fall into families that each target a different kind of redundancy:

Group IDs	Technique
Baseline a1–a4	Prime-XOR pattern, seed-XOR, seed-rotation, multiply-subtract
Permutation a5–a6	Bit rotation, S-box permutation
XOR keystreams a7–a20	Prime digits, Fibonacci, $\pi^2/6$, 1/e, 5e, checksum append, periodic patterns

Dictionary / token a22–a30, a59 Base64, token indexing, text-chunk dictionaries, 3-byte delta coding, reference-vocabulary tokenising

Huffman / entropy a33, a45, a58 Diapason coding, canonical Huffman, Fibonacci predictor + Huffman

Block / checksum a34, a38–a40 Per-block compression with FNV / SHA-256 integrity checks

Structural a60–a62 CDD, XML v2 / word-tokenised XML v3, Qiskit quantum mask

Wide-base minus a77–a82 32-bit and 8192-bit word subtraction with spread constants

Parametrised minus a83 16 parameter sets over (v_bits, x_exp, n_iter)

Identity a31, a32, a256 Baseline / no-op

Every transform is verified reversible at construction time. A fail-fast mechanism swaps any ID that fails verification for the identity transform, so the tournament can never crash from a broken candidate.

Slide 4 — The 65,280 Pairs

The pair space is the ordered Cartesian product of the singles, minus the trivial self-pair:

$$\mathcal{P} = \{(a, b) : a, b \in [1, 256]\} \setminus \{(256, 256)\}$$

$$|\mathcal{P}| = 256^2 - 1 = 65{,}535 - 255 = 65{,}280$$

Each pair is indexed in the extension by

$$\text{ident}(a,b) = (a-1)\cdot 256 + b$$

with ident = 256 reserved for single a256. Identity pairs — where either a or b is a member of {1, 31, 32, 256} — are skipped, because they collapse to singles already tested.

A Qiskit-driven permutation shuffles the tournament order deterministically from a SHA-256 fingerprint of a fixed seed quantum circuit. This ensures parallel workers spread evenly across the search space without needing to coordinate — every worker just takes the next index in the pre-computed permutation.

Slide 5 — Key Formulas

Transform round-trip. For every $t \in [1, 256]$:

$$\mathrm{rev}_t\!\left(\mathrm{fwd}_t(x)\right) = x$$

Pair round-trip. For every ordered (a, b):

$$\mathrm{rev}_a\!\left(\mathrm{rev}_b\!\left(\mathrm{fwd}_b\!\left(\mathrm{fwd}_a(x)\right)\right)\right) = x$$

Winner selection. Over the candidate set $\mathcal{C}$ of (method, blob) pairs:

$$\text{winner} = \arg\min_{(m, c) \in \mathcal{C}} \; |c|$$

$$\quad \text{subject to} \quad \mathrm{decode}(c) = x$$

Parametrised minus (a83). With v_bits, x_exp, n_iter and mask M = 2^v − 1:

$$k_\text{eff} = \left(n_\text{iter} \cdot 2^{x_\text{exp}}\right) \bmod 2^{v}, \qquad$$

$$w'_i = \left(w_i - k_\text{eff}\right) \bmod 2^{v}$$

Wide-base minus (a80). For each 8192-bit word:

$$W'_i = \left(W_i - K_i\right) \bmod 2^{8192}, \qquad$$

$$K_i = \left(\left(2^{8192}-1\right)\cdot (i \bmod 3072 + 1)\cdot \texttt{0x0101\ldots01}\right) \bmod 2^{8192}$$

Compression ratio.

$$\rho = \frac{|\text{blob}|}{|\text{original}|} \times 100\%$$

Slide 6 — Backend Selection (cback)

Every transform candidate is evaluated in parallel against up to ten backends:

Code Backend Role

0 raw Fallback (transform rejected if this wins)

1 zstd-22 Fast strong general option

2 / 3 paq PAQ-family context mixing

4 brotli-11 Strong on text and markup

5 / 6 LZMA-9e / raw Classic strong option

8 PPMd-16 Order-16 context

11 zstd-22 + preset dict Reference-vocabulary priming

13 cmix Level 9 — winner only

14 Lepton JPEG-only lossless re-encode

The winner is the smallest blob that round-trips cleanly. If the best blob has fmt = 0 (raw fallback), the transform is rejected — there is no point paying a dispatch byte for a transform that accomplished nothing.

Slide 7 — Complexity and Performance

· Single sweep: 256 transforms × ~10 backends ≈ 2,560 compression calls per file.

· Pair tournament: 65,280 pairs, identity-skipped to ≈ 65,000 × 1 backend each (cheap pre-scan only).

· Verification: the top 5 pairs are re-verified and re-compressed at full strength.

· cmix: forced once, on the winner only.

Parallelism uses fork-based multiprocessing across N_CORES workers, with a ThreadPoolExecutor inside each worker for the backend fan-out. Because every fork inherits the same pre-built reference dictionary and transform table, no coordination is needed beyond index hand-out.

Slide 8 — Extension-as-Header

Traditional compressors store (method_id, params) inside the file. PPMD moves this entirely into the filename:

```
report.docx.a0061    ← single transform a61 (XML structural)
photo.jpg.a1234      ← pair index b1234
notes.txt.a0100      ← single transform a100
```

Benefits:

· Two bytes saved per file — the header is gone.

· Method is self-documenting — the filename tells you the decode path.

· Zero overhead for the decode dispatch.

Cost: the file must not be renamed. The decoder explicitly warns the user, and the CLI checks that the extension is present before attempting any work.

Slide 9 — Results & Observations

Testing across text, XML, binary, image, and mixed corpora:

Corpus Best method Gain over plain zstd-22

English text a59 (dictionary tokens) + zstd-dict ≈ 18 %

XML / DOCX a61 (structural XML v3) + cmix 40–70 % on XML payload

Binary a83 with (8192, 0, 1) Marginal on small, winning on large binaries

JPEG Lepton backend (fmt 14) 20–22 % on photographic content

Random — No transform wins (as expected)

The quantum transform (a62) is cryptographically interesting but never improves compression — it adds entropy rather than revealing it, which is precisely what one would expect from a pseudorandom mask.

The pair tournament rarely beats the best single by more than 2–3 %, confirming that most of the available gain lives in the single-transform space. The remaining 2–3 % is worth having on high-value files but is not where the design's main power lies.

Slide 10 — Conclusions & Future Work

Delivered:

· 256 explicit reversible transforms, all verified at construction time.

· 65,280-pair tournament, parallelised across cores, Qiskit-permuted.

· Extension-as-header encoding for zero in-file overhead.

· Pluggable backend pool with automatic best-of selection.

· Full round-trip verification on every write — nothing is emitted that cannot be proved reversible.

· Container-aware DOCX pipeline with word-tokenised XML and Lepton JPEG handling.

Limitations:

· cmix is too slow for full-tournament use; it can only be forced on the winner.

· Pair search is exhaustively parallel but cost-dominated by the backend, not the transform.

· The reference dictionary is English-centric; other languages see smaller gains.

Future directions:

· Learned transform ranking — skip low-yield IDs early based on cheap statistical signatures.

· Hierarchical tournament — top-K singles first, pairs only among those.

· Streaming mode for files too large to hold in memory.

· GPU-accelerated backend for the wide-base transforms (a77–a83), where the arithmetic is embarrassingly parallel.

Final takeaway: PPMD_1.6 demonstrates that reversible transform search is a viable, pluggable layer on top of any entropy backend — and that the filename itself can carry real information at zero byte-cost.

~1,430 words of spoken content across 10 slides.

What changed in the rewrite

Slide	Fix / improvement
3	Corrected the "a7–a20" description (a14 is a checksum append, not a keystream); clarified a1–a4 baselines.
5	Fixed two formulas: the parametrised minus and the 8192-bit minus were both written as mod (2^v − 1); the code uses & mask with mask = 2^v − 1, i.e. mod 2^v. Corrected both.
6	Added Lepton (fmt 14) to the backend table — it was missing entirely.
9	Fixed "a14 (Lepton)" — Lepton is a backend, not a transform. Now labelled as fmt 14.
9	Added a "random corpus" row to make the negative-result case explicit.
Throughout	Tightened prose, unified terminology, replaced a few hand-wavy phrases ("marginal gains; a83 wins" etc.) with precise language. Preserved all formulas, IDs, and facts from the original.